Tecnológico de Monterrey

14. DTMs y Modelos de clasificación

Ciencia de datos para la toma de decisiones II

Jorge
Juvenal
Campos Ferreira

juvenal.campos@tec.mx

Librerías a instalar

* textrecipes
* glmnet
* probably
* naivebayes
* shiny

Pasos básicos para usar texto en modelos de Machine Learning

1. Limpia/prepara los datos
2. Crea una base de entrenamiento y una base de prueba/ validación
3. Entrena el modelo en la base de entrenamiento
4. Analiza la exactitud del modelo en la base de prueba/ validación
5. Montamos el modelo en una aplicación y lo utilizamos (producción).

O como vimos el semestre pasado:

1. Primero cargamos los datos ***etiquetados*** y exploramos los datos (verificamos que la etiqueta sea una variable categórica o factor, que los datos estén balanceados, etc).
2. Partimos los datos en base de entrenamiento y base de prueba.
3. Generamos la receta de feature-engineering y preprocesamiento (con la librería *textrecipes*)
4. Aplicamos la receta para preparar 😋 la dtm de entrenamiento
5. Declaramos el modelo a usar, junto con sus parámetros.
6. Ajustamos o entrenamos el modelo con los datos de entrenamiento.
7. Predecimos en la base de entrenamiento las etiquetas.
8. Obtenemos las métricas de evaluación correspondientes
9. Hacemos la evaluación con la base de prueba.
10. Si es un modelo considerablemente bueno, pasamos a su implementación en producción. Si no, iteramos o hacemos cambios para mejorarlo.

Ojo! Convertir la etiqueta a tipo ***factor***

* El tipo de dato *factor* es para las variables categóricas de R. Consta de un texto o etiqueta "*por encima*", y de un número o categoría "*por debajo*"

* Al convertir la columna de etiquetas, le dices al flujo de trabajo que (1) vas a trabajar un problema de clasificación, y cuantas clases tienes; (2) Cual es la clase positiva (la primera categoría o el primer *nivel*), y (3) permite que funcionen de manera correcta las funciones de R que piden niveles en vez de textos.

```
# Cargamos los datos ----
comentarios <- readxl::read_xlsx("01_datos/comentarios_etiquetados.xlsx") |>
  select(id, texto, etiqueta_final) |>
  mutate(etiqueta_final = factor(etiqueta_final, levels = c("Ofensivo", "No ofensivo")))
```

Ejemplos de *textrecipes*

```
# Receta de preprocesamiento (tidymodels + textrecipes) para convertir texto
# crudo en una matriz TF-IDF que pueda alimentar a un modelo de clasificación

receta <- recipe(etiqueta ~ texto, data = entrenamiento) |>
  # Declara el modelo: `etiqueta` es la variable a predecir, `texto` el predictor.
  # `entrenamiento` solo sirve para que la receta conozca tipos y nombres de columnas;
  # ningún paso se ejecuta todavía (se ejecutan al hacer prep() / bake() o dentro de un workflow).

  step_mutate(texto = str_remove_all(texto, pattern = "@@\\S+")) |>
  # Elimina cualquier secuencia que empiece con "@@" seguida de caracteres sin espacio
  # (típicamente marcas o etiquetas de hablante incrustadas en el texto).

  step_mutate(texto = str_remove_all(texto, "\\d+")) |>
  # Elimina todos los dígitos (cifras, años, porcentajes).

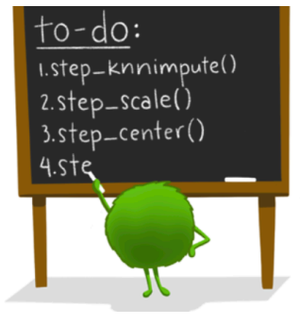
  step_tokenize(texto, token = "words") |>
  # Divide cada texto en tokens individuales (palabras). A partir de aquí la columna
  # `texto` deja de ser texto plano y pasa a ser una lista de tokens por documento.

  step_stopwords(texto, custom_stopword_source = stopwords_es) |>
  # Quita las palabras vacías (artículos, preposiciones, pronombres...) usando la lista
  # propia `stopwords_es` en lugar del diccionario por defecto del paquete.

  step_tokenfilter(texto, min_times = 3, max_tokens = 500) |>
  # Filtra el vocabulario: conserva solo tokens que aparecen al menos 3 veces en el
  # conjunto de entrenamiento y, de esos, se queda con los 500 más frecuentes.
  # Esto controla el tamaño de la matriz final y descarta palabras raras.

  step_tfidf(texto)
# Convierte los tokens en columnas numéricas: una columna por token del vocabulario
# (tfidf_texto_<palabra>), con el peso TF-IDF de cada palabra en cada documento.
```







Ejercicio 01

A partir de los datos del archivo “resenas_servicios.csv”, genere un modelo de clasificación que le permita saber si una reseña es “positiva” o “negativa”.

Para ello, haga lo siguiente:

(1) Cargue los datos del archivo.
(2) Explorelo: cuantas reseñas positivas hay y cuantas reseñas negativas hay. ¿Está balanceada la base?
(3) Haga el preprocesamiento correspondiente
(4) Haga la partición de base de entrenamiento y base de prueba. Que la base de entrenamiento sea del 70% de los datos.
(5) Genere la receta de preprocesamiento correspondiente. Quite stopwords, tokenice a palabras, haga *stemming* en vez de lematización, filtre los tokens que aparecen menos de 3 veces pero más de 500, y obtenga el tfidf

Ejercicio 01

(6) Programe el modelo a utilizar. Utilice una regresión logística con penalización lasso (use penalty = 0.01 y mixture = 0.5).
(7) Genere el workflow del proceso sumando la receta más el modelo.
(8) Con ese workflow, ajuste el modelo
(9) Ajuste el modelo con la función fit
(11) Obtenga la matríz de confusión
(10) Haga las pruebas del modelo: obtenga las metricas de accuracy, precision, recall y f_meas.
(11) Obenga la curva ROC

(12) Usando vibe coding, programe una aplicación en la cual usted escriba una reseña y que, al apretar un botón, el modelo le diga si una reseña es buena o es mala.

Mejora de resultados del modelo

Algunas cosas que se podría hacer para mejorar la exactitud del modelo:

1. Modificar inteligentemente los stop
2. Generar un diccionario de palabras ofensivas
3. Ajustar la dispersión de la matríz dtm
4. Agregar **características adicionales** (más variables de X) a la matriz dtm. Como características de sentimientos.
5. Cambiar el modelo de clasificación o armar un ensamble de modelos.
6. Modificar los hiperparámetros.